

VIEDIT 1.0 — User Manual

A **vi**-style full-screen text editor for CP/M 2.2.

VIEDIT brings the modal editing model of Bill Joy's **vi** to an 8-bit CP/M system. It runs on any CP/M 2.2 machine with enough free TPA (Transient Program Area) — it is not tied to a particular memory size. It drives any VT100-compatible terminal, edits files entirely in RAM for speed, and preserves your file's exact byte layout (including stray carriage returns and missing final newlines) on save.

1. Quick Start

```
A>VIEDIT MYFILE.TXT
```

This opens **MYFILE.TXT** for editing. If the file does not exist, VIEDIT starts with an empty buffer and creates the file when you save.

You begin in **command mode**. To make changes you switch to **insert mode**, type your text, then press **ESC** to return to command mode. To save and quit:

```
:wq          (then press Return)
```

or, equivalently, type **ZZ** in command mode.

To quit **without** saving:

```
:q!          (then press Return)
```

If you remember nothing else, remember these five things:

You want to...	Do this
Move the cursor	h j k l (left, down, up, right)
Start typing text	i (insert), then type
Stop typing text	press ESC
Save and quit	:wq Return, or ZZ
Quit, throwing away edits	:q! Return

2. The Two Modes

VIEDIT is **modal**. The same keys mean different things depending on the mode.

- **Command mode** (the default). Keystrokes are *commands* — move the cursor, delete text, change text, search, save. Letters do **not** appear on screen as text; they act.
- **Insert mode**. Keystrokes are *text* — what you type is inserted into the file. Press `ESC` to leave insert mode and return to command mode.

The status line at the bottom of the screen shows `-- INSERT --`, `-- REPLACE --` when you are in one of those modes. A blank status area means you are in command mode.

If you are ever unsure which mode you are in, press `ESC`. From command mode `ESC` does nothing harmful; from any other mode it returns you to command mode.

3. Starting VIEDIT

3.1 Command line

```
VIEDIT [filename] [/Ln] [/Cn] [/R]
```

Argument	Meaning
<code>filename</code>	File to edit. Omit it to start with an empty, unnamed buffer.
<code>/Ln</code>	Use <code>n</code> screen lines (rows). Clamped to 24–200.
<code>/Cn</code>	Use <code>n</code> screen columns . Clamped to 80–132.
<code>/R</code>	Open read-only — <code>:w</code> and <code>ZZ</code> refuse to write.

Examples:

```
A>VIEDIT REPORT.TXT      edit REPORT.TXT at the default 80x24
A>VIEDIT REPORT.TXT /L43  edit on a 43-line terminal
A>VIEDIT BIOS.ASM /R      view a file read-only
A>VIEDIT                  start with an empty buffer
```

3.2 Configuration file — `VIEDIT.CFG`

If a file named `VIEDIT.CFG` exists on the current drive, VIEDIT reads your default screen geometry from it at startup. It is a plain text file with `keyword=value` lines:

```
lines=43
columns=80
```

The command-line switches (`/Ln`, `/Cn`) override the config file. Geometry may only **enlarge** past the 80×24 minimum; values are clamped to 24–200 rows and 80–132 columns.

3.3 Automatic terminal-size detection

If you give **neither** a row nor a column count (no `/Ln` / `/Cn` switch and no `lines=` / `columns=` line in `VIEDIT.CFG`), VIEDIT asks the terminal how big it is at startup. It parks the cursor at the bottom-right corner and issues the VT100 cursor-position report query (`ESC[6n`); a VT100-compatible terminal answers with its actual size, and VIEDIT adopts it (still clamped to 24–200 rows and 80–132 columns). Whichever dimension you *did* specify is left alone, so `VIEDIT /C80` keeps 80 columns but still auto-detects the row count.

A terminal that does not support the query simply stays silent; VIEDIT waits briefly, then falls back to the 80×24 default (or whatever you set). The wait is short, so startup is never noticeably delayed.

3.4 Memory limit

VIEDIT edits entirely in RAM, which keeps every operation fast. The editable file size therefore depends on how much TPA your CP/M system provides: the larger the TPA, the larger the file you can open. A file that will not fit in the available buffer is refused up front with:

```
File too large to edit in available memory
```

and you are returned to CP/M. This is deliberate: a file that does not fit is better refused than opened and left sluggish.

4. Moving Around

All movement commands are typed in command mode. Many accept a **count** prefix — type a number first to repeat the motion (e.g. `5j` moves down five lines, `3w` moves forward three words).

4.1 Character and line motion

Key	Moves the cursor...
<code>h</code> / <code>l</code>	left / right one character
<code>j</code> / <code>k</code>	down / up one line (keeps its goal column)
<code>0</code>	to the first column of the line
<code>^</code>	to the first non-blank character of the line
<code>\$</code>	to the last character of the line
Arrow keys	same as <code>h</code> <code>j</code> <code>k</code> <code>l</code>
Home / End	same as <code>0</code> / <code>\$</code>

4.2 Word motion

Key	Moves...
<code>w</code>	forward to the start of the next word
<code>b</code>	back to the start of the previous word
<code>e</code>	forward to the end of the current/next word
<code>W</code> <code>B</code> <code>E</code>	same, but a WORD is whitespace-delimited (punctuation counts as part of the word)

A *word* is a run of letters, digits, and underscores; punctuation forms its own words. A *WORD* (capital) is any run of non-blank characters.

4.3 Find a character on the line

Key	Moves to...
<code>f{c}</code>	the next occurrence of character <code>c</code> (cursor lands on it)
<code>F{c}</code>	the previous occurrence of <code>c</code>
<code>t{c}</code>	just before the next <code>c</code>
<code>T{c}</code>	just after the previous <code>c</code>
<code>;</code>	repeat the last <code>f</code> / <code>F</code> / <code>t</code> / <code>T</code>
<code>,</code>	repeat it in the opposite direction

4.4 Jump within the file

Key	Jumps to...
gg	the first line of the file
G	the last line of the file
nG	line number <code>n</code> (e.g. <code>42G</code> goes to line 42)
%	the bracket matching the one under the cursor (<code>()</code> , <code>[]</code> , <code>{}</code>)

4.5 Jump within the screen

These move the cursor without scrolling — they land on a line that is already displayed.

Key	Jumps to...
H	the top line of the screen ("high")
M	the middle line of the screen
L	the bottom line of the screen ("low")
nH	<code>n</code> lines down from the top (<code>3H</code> = third line shown)
nL	<code>n</code> lines up from the bottom

4.6 Scrolling

Key	Scrolls...
^F	forward one full screen (page down)
^B	back one full screen (page up)
^D	down half a screen
^U	up half a screen
^L	redraw the screen (does not move the cursor)

(`^F` means hold Control and press `F` .)

5. Inserting Text

Each of these switches to insert mode. Type your text, then press `ESC` to return to command mode.

Key	Begins inserting...
i	before the cursor
a	after the cursor (append)
I	at the first non-blank of the line
A	at the end of the line
o	on a new line below the current line
O	on a new line above the current line

While in insert mode:

- Printable characters and `Tab` are inserted.
- `Return` starts a new line.
- `Backspace` deletes the character to the left.
- `ESC` returns to command mode.

6. Changing and Deleting Text

6.1 Single-character edits

Key	Action
x	delete the character under the cursor
r{c}	replace the character under the cursor with <code>c</code>
~	switch the case of the character, and move right
J	join the next line onto the end of the current line
R	enter replace mode — typing overwrites; <code>ESC</code> exits

`x` and `r` accept a count: `3x` deletes three characters.

6.2 Operators + motion

The operators `d` (delete), `c` (change), and `y` (yank/copy) combine with any motion to act on the text the motion would cover:

Type...	To...
dw	delete to the start of the next word
de	delete to the end of the current word

Type...	To...
d\$	delete to the end of the line
d0	delete to the start of the line
cw	change a word (delete it and enter insert mode)
c\$	change to the end of the line
yw	yank (copy) a word

The pattern is **operator + motion**. Counts work too: `2dw` deletes two words, `3dd` deletes three lines. `c` leaves you in insert mode after deleting.

6.3 Whole-line operators

Key	Action
dd	delete the current line
yy	yank (copy) the current line
cc	change the whole line (clear it, enter insert)
D	delete from the cursor to the end of the line
C	change from the cursor to the end of the line
S	change the whole line (same as <code>cc</code>)
s	substitute — delete the character, enter insert

Line operators are *linewise*: `5dd` deletes five lines as a block.

Yank and delete copy the affected text into a buffer (so it can be put back with `p`). That buffer holds about **1 000 bytes** line-wise (**250** for a character-wise range such as `y`a`). A range that would overflow it is refused with **Range too large** and nothing is changed — it is never silently truncated.

6.4 Indenting

Key	Action
>>	shift the current line right one Tab
<<	shift the current line left one Tab
n>>	shift <code>n</code> lines right

With a mark or motion `>` and `<` act as operators too (e.g. `>'a` shifts the lines through mark `a`); see §9.

7. Copy and Paste (Yank and Put)

VIEDIT keeps one **unnamed register**. Deleting (`x` , `dd` , `dw` , ...) and yanking (`yy` , `yw` , ...) both fill it; `p` and `P` paste it back.

Key	Action
<code>yy</code>	yank the current line into the register
<code>nyy</code>	yank <code>n</code> lines
<code>p</code>	put the register after the cursor (or below, if linewise)
<code>P</code>	put the register before the cursor (or above, if linewise)

A linewise yank (`yy` , `dd`) pastes as whole lines above/below; a charwise yank (`yw` , `x`) pastes inline. So `dd` then `p` moves a line down one position, and `yy` then `p` duplicates a line.

8. Undo and Repeat

Key	Action
<code>u</code>	undo the last change
<code>.</code>	repeat the last change

`u` is **single-level** — it undoes the most recent change. Pressing `u` again does not undo further back. If there is nothing to undo, VIEDIT shows `Nothing to undo`.

`.` replays your last edit. For example, after `cwfoo + ESC` (change a word to "foo"), moving to another word and pressing `.` changes that word to "foo" too.

9. Marks

A *mark* remembers a place in the file. Set a mark, move anywhere, and jump back to it instantly — or use it as the far end of a range for an operator.

Key	Action
<code>m{a-z}</code>	set mark <code>a-z</code> at the cursor
<code>`{a-z}</code>	jump to the exact position of a mark (line and column)
<code>'{a-z}</code>	jump to the line of a mark (first non-blank character)

Marks move with the text: insert or delete above a mark and it still points at the same character.

Marks as ranges. Any operator (`y` , `d` , `c`) can take a mark as its motion:

Example	Acts on...
<code>y'a</code>	yank the lines from the cursor through mark <i>a</i> (line-wise)
<code>d'a</code>	delete those lines
<code>c'a</code>	change those lines
<code>y`a</code>	yank the characters from the cursor to mark <i>a</i> (char-wise)
<code>>'a <'a</code>	shift those lines right / left

This is the classic vi way to select a region: go to one end and set a mark (`ma`), move to the other end (with searches, `^F` / `^B` paging, `G` , ...), then operate (`y'a` , `d'a`). It is far faster than nudging a highlight one line at a time — and just as quick on a slow serial terminal.

Marks also work as **ex ranges**: `:'a, 'b s/old/new/g` substitutes only the lines from mark *a* through mark *b* (see §11).

A mark range (or any `Nyy` / `Ndd`) that will not fit the yank/undo buffer (about 1 000 bytes line-wise, or 250 for a ``` character range) is refused with **Range too large** rather than silently truncated, so a copy or move is never quietly cut short.

10. Searching

Key	Action
<code>/text</code>	search forward for <code>text</code> (press Return)
<code>?text</code>	search backward for <code>text</code> (press Return)
<code>n</code>	repeat the last search in the same direction
<code>N</code>	repeat the last search in the opposite direction
<code>*</code>	search forward for the word under the cursor
<code>#</code>	search backward for the word under the cursor

`*` and `#` save you retyping: put the cursor on a word (letters, digits, and underscores) and press `*` to jump to its next occurrence. If the cursor is not on a word, the next word to its right on the line is used. Both set the search pattern, so `n` and `N` continue from there. The match is literal, so `*` on `count` also matches `counter` .

Searches are literal (no wildcards or regular expressions) and **wrap around** the file once. If the text is not found, VIEDIT shows `Pattern not found` and leaves the cursor where it was.

At the `/` or `?` prompt, `Backspace` erases; backspacing past the prompt cancels the search.

11. Search and Replace

The `:s` (substitute) command finds text and replaces it, optionally across a range of lines. The general form is `:[range]s/old/new/[flags]`.

```
:s/old/new/      replace the first "old" on the current line
:s/old/new/g      replace every "old" on the current line
:%s/old/new/g     replace every "old" in the whole file
:.,$s/old/new/g   replace from the current line to the end of the file
:1,5s/old/new/    replace the first "old" on each of lines 1-5
```

Ranges go before the `s`. An address is `.` (current line), `$` (last line), `'a` (the line of mark `a`, see §9), or a line number:

Range	Lines affected
<code>(none)</code>	the current line only
<code>%</code>	the whole file (same as <code>1, \$</code>)
<code>., \$</code>	the current line through the last line
<code>N, M</code>	lines <code>N</code> through <code>M</code>
<code>'a, 'b</code>	from mark <code>a</code> through mark <code>b</code>

Flags go after the closing delimiter:

Flag	Effect
<code>g</code>	replace all matches on each line, not just the first

Notes:

- Matching is **literal** — the same as `/` search, with no wildcards or regular expressions.
- An empty `old` reuses the **last search pattern**: `/foo` then `:%s//bar/g` changes every `foo` to `bar`. A bare `:s` repeats the previous substitution.
- A `:s` also **sets the search pattern**, so after replacing the first match you can press `n` to jump to the next occurrence (and `N` for the previous).

- The `new` part may be **empty** to delete the matches: `:%s/foo//g`.
- The delimiter is whatever character follows the `s`, so you can avoid escaping a literal `/` by choosing another one: `:s#a/b#c#` turns `a/b` into `c`. Within the default `/` delimiter, escape a literal slash as `\/`.
- When finished, VIEDIT reports `N subs, M lines` — the number of replacements and the number of lines changed. If nothing matched, it shows `Pattern not found` and leaves the file unchanged.
- A whole `:s` (even a file-wide `:%s/.../.../g`) is reversed by a single `u`.

12. File Commands (Ex Commands)

Type `:` in command mode to bring up the ex prompt at the bottom of the screen, then a command, then Return.

Command	Action
<code>:w</code>	write (save) the file
<code>:w file</code>	write to a different file (save-as)
<code>:wq / :x</code>	write and quit
<code>:q</code>	quit — refuses if there are unsaved changes
<code>:q!</code>	quit, discarding unsaved changes
<code>:r file</code>	read <code>file</code> in below the current line
<code>:e file</code>	edit a different file (refuses if there are unsaved changes)
<code>:e! file</code>	edit a different file, discarding unsaved changes
<code>:e</code>	show the current file's status (name, lines, bytes)
<code>:s/old/new/</code>	search and replace — see §11
<code>:h / :help</code>	show the help screen
<code>:stat</code>	show buffer/memory diagnostics

Notes:

- `:q` on a modified buffer reports `No write since last change (add ! to override)` and refuses. Use `:q!` to discard, or `:wq` to save first. `:e!` likewise discards a modified buffer and loads the new file.
 - `:w` on a read-only file (`/R`) reports `File is read only`.
 - At the `:` prompt, `Backspace` erases; backspacing past the prompt cancels.
-

13. Status and Help

Key	Shows...
<code>^G</code>	the file status: "NAME" line N of M col C
<code>K</code>	the full command-summary help screen (press any key to exit)
<code>^L</code>	redraw the screen

The help screen (`K`, or `:h`) is a one-page reference for every command. It is the fastest way to jog your memory without leaving the editor.

14. Notes on File Format

- **Tabs** are displayed expanded to 8-column stops, but the real Tab character is preserved in the file. The cursor column reflects the displayed position.
 - VIEDIT preserves your file **byte-for-byte** on save, including lone carriage returns, bare line feeds, and a missing final newline. What you open is what you save (apart from your edits).
 - New files and saves use standard CP/M `CR`, `LF` line endings and a trailing `^Z` (end-of-file marker) pad in the last record, as other CP/M tools expect.
-

15. Command Summary (Quick Reference)

```
MOVE  h j k l  0 ^ $      w b e  W B E
      f F t T  ; ,      gg G   nG
      H M L screen top/mid/bottom
      ^F ^B page  ^D ^U half  %  match bracket

INSERT i a I A   o O open line  R replace mode

EDIT   x del    r{c} replace  ~ case  J join
      d c y + motion  dd yy cc  D C S s
      >> << indent   p P put   u undo   . repeat

FIND   / ? search   n N next/prev  * # word under cursor
SUBST  :s/old/new/[g]  :%s whole file  :.,,$s to end  :N,Ms / : 'a, 'b range

MARK   m{a-z} set    `a exact / 'a line jump  y'a d'a c'a range

MISC   ^G status    ^L redraw    K help

FILE   :w write      :w file save-as  :r file read    :e file edit
EXIT   :q quit       :q! force        :wq :x ZZ  write+quit
```

VIEDIT 1.0 — a vi-style editor for CP/M 2.2.